

4.1 Project Overview

Objective

Design and implement a complete Enterprise ETL solution using SQL Server Integration Services (SSIS) to extract, validate, transform, and load operational retail data into a Retail Data Warehouse, supporting both Full Load and Incremental Load processes.

Business Problem

Retail companies generate large volumes of operational data across multiple business processes — sales, orders, inventory, and financial transactions. This data, in its raw form, is not structured for analytical reporting. An ETL pipeline is required to validate, cleanse, enrich, and consolidate this data into a form suitable for business intelligence and decision-making.

Data Source & Scope Justification

The source data for this project is the **ContosoRetailDW** database. It is important to note that this source is not a raw operational (OLTP) system, but rather an existing dimensional data warehouse containing pre-structured Fact and Dimension tables. For the purpose of this project, these tables were treated as the operational source system, and a complete independent ETL pipeline (Staging → Validation → Business Transformation → Data Warehouse) was designed and built around them, rather than reusing the source's existing structure as-is.

Four Fact tables were selected to represent the core business processes required by the project brief:

- **FactSales** — in-store sales transactions (also covers Orders/Purchases, since the source does not maintain a separate Orders process)
- **FactOnlineSales** — online sales transactions, linked to individual customers
- **FactInventory** — daily inventory snapshots, representing Stock Movements
- **FactExchangeRate** — daily currency exchange rates, representing financial Transactions

FactITSLA and **FactITMachine** were excluded, as they represent IT infrastructure operations rather than retail business processes.

Ten Dimension tables were included based on direct foreign key relationships to the four Facts: **DimDate**, **DimStore**, **DimProduct**, **DimProductSubcategory**, **DimProductCategory**, **DimPromotion**, **DimCurrency**, **DimChannel**, **DimCustomer**, and **DimGeography**. Other available dimensions (DimAccount, DimScenario, DimEmployee, DimEntity, DimSalesTerritory, DimMachine, DimOutage) were excluded as they are unrelated to the selected Facts or belong to the excluded IT/financial-reporting subject area.

Solution Architecture

Source (ContosoRetailDW) → Staging → Validation → Business Transformation → Data Warehouse (dw) → Reporting

Each stage is implemented as an independent SSIS package, writing to its own dedicated schema (staging, validation, transformation, dw), so that every stage of the data's lifecycle is independently inspectable and re-runnable.

4.2 ETL Architecture Diagram

ContosoRetailDW | —▶ | staging | —▶ | validation | —▶ | transformation | —▶ | dw |

- **Staging:** Raw 1:1 copy of the 14 source tables, no business logic applied.
 - **Validation:** Mandatory field, numeric, and business-rule checks applied per table; produces _Valid/_Invalid outputs plus a shared technical error log.
 - **Business Transformation:** Financial calculations, business classifications, data standardization, and audit trail applied to the validated data.
 - **Data Warehouse (dw):** Final star-schema with real Primary Keys and Foreign Key relationships between Facts and Dimensions, ready for reporting.
 - **Incremental Load:** A separate pass that re-runs the pipeline and applies Lookup-based Insert/Update logic against the dw schema for subsequent executions.
-

4.3 Package Documentation

Package 1 — Staging_Area.dtsx

Purpose	Raw copy of all 14 source tables (4 Facts + 10 Dims) from ContosoRetailDW into the staging schema
Input	ContosoRetailDW source tables (dbo schema)
Output	staging.<TableName>_Stg (14 tables)
Business Logic	None — raw copy only, per project requirement (Section 3.2)
Control Flow	Execute SQL Task (Truncate all 14 staging tables) → parallel Sequence Containers for Dimensions and Facts, each containing one Data Flow Task per table using OLE DB Source → OLE DB Destination (Fast Load)

Package 2 — ValidationArea.dtsx

Purpose	Apply 3-stage validation (Mandatory Fields → Numeric Values → Business Rules) to all 14 staged tables
Input	staging.<TableName>_Stg

Output	validation.<TableName>_Valid, validation.<TableName>_Invalid, shared dbo.Pipeline_Errors
Business Logic	See Section 4.4
Control Flow	Execute SQL Task (Truncate 28 validation tables + Pipeline_Errors) → Facts_Validation and Dims_Validation Sequence Containers, each with one Data Flow Task per table

Package 3 — Transformation.dtsx

Purpose	Apply financial calculations, business classifications, data standardization, and audit trail to validated data
Input	validation.<TableName>_Valid
Output	transformation.<TableName>_Transformed (14 tables)
Business Logic	See Section 4.5
Control Flow	Execute SQL Task (Truncate) → Facts_Transformation and Dims_Transformation Sequence Containers

Package 4 — Full_Load_DWH.dtsx

Purpose	Populate the final dw schema (star schema with real PK/FK constraints) from the transformation layer
Input	transformation.<TableName>_Transformed (a curated column subset — several near-100%-NULL and legacy columns excluded, see Section 4.7)
Output	dw.<TableName> (14 tables, no suffix)
Business Logic	Column selection/exclusion only — no further calculation
Control Flow	Execute SQL Task (Truncate dw tables in strict FK-dependent order: Facts first, then dependent Dims, then independent Dims) → Dimension loads (in FK dependency order: Geography/Currency/Date/ProductCategory/Channel → ProductSubcategory/Store/Customer → Product → Promotion) → Fact loads

Package 5 — Incremental_Load.dtsx

Purpose	Detect new vs. existing records and apply Insert/Update logic against dw on subsequent pipeline runs
Input	transformation.<TableName>_Transformed

Output dw.<TableName> (updated in place)

Business Logic See Section 4.6

Control Flow One Data Flow Task per table: OLE DB Source → Lookup Transformation (on Primary Key) → Match Output (OLE DB Command / UPDATE) + No Match Output (OLE DB Destination / INSERT) + Error Output (technical errors → Pipeline_Errors)

4.4 Data Validation Documentation

Validation Rules

Each of the 4 Fact tables is validated through three sequential stages via a Conditional Split, in this fixed order: **Mandatory Fields → Numeric Values → Business Rules**. Each Dimension table receives a simplified validation limited to Mandatory Fields, with Numeric and Business Rule checks added only where the table contains relevant numeric or date-logic attributes (e.g., DimPromotion, DimStore, DimProduct, DimCustomer).

Rules were not assumed from the schema alone: staged data was profiled with aggregate NULL-count queries per column, and any nominally-nullable column found to be always populated in practice was reclassified as mandatory, simplifying the corresponding checks.

Error Handling Strategy

Every Conditional Split branch that fails validation is tagged with a Derived Column identifying the failure type (Missing Mandatory Field, Invalid Numeric Value, Business Rule Violation) before being written to that table's _Invalid output — records are never dropped silently, and the pipeline continues processing regardless of rejected rows (Requirement 6 — Pipeline Continuity).

Separately, technical/structural failures (e.g., data type mismatches) are captured via each component's built-in Error Output and redirected to a single shared table, distinguishing business-rule rejections from genuine pipeline errors.

Error Logging Approach

- validation.<Table>_Invalid — business-rule rejections, with ErrorType and ErrorDate.
- dbo.Pipeline_Errors — shared technical error log across all 14 tables (PipelineErrorID, SourceTable, ErrorColumn, ErrorDescription, ErrorCode, ErrorDate), populated via each package's Union All of Error Outputs.

Across all validation runs, Pipeline_Errors remained empty, confirming the source data — being an already-structured warehouse — contained no type-mismatch issues; all rejections were genuine business-rule violations.

4.5 Business Transformation Documentation

(Full detail in the separately-delivered Transformation Layer Documentation. Summary below.)

- **Financial Calculations (Facts):** GrossProfit, NetSalesAmount, ProfitMarginPercent, AverageUnitProfit (FactSales, FactOnlineSales); InventoryValue, OnOrderValue, StockCoverageDays (FactInventory); RateVariance, RateVariancePercent (FactExchangeRate) — all division operations protected against divide-by-zero.
 - **Business Classifications:** distributed across whichever table holds the relevant attribute — StockStatus/RateTrend (Facts); PromotionStatus/DiscountTier, StoreOperationalStatus/StoreSizeCategory, PriceTier/ProductAvailability, IncomeSegment/CustomerLifeStage (Dimensions).
 - **Data Standardization:** Proper Case for customer names, phone number formatting (dash/parenthesis removal), trimming and uppercasing of category/label text fields across Dimensions.
 - **Audit Information:** LoadDate, PackageName, ETLUser, BatchID appended to all 14 tables, with BatchID generated once per package execution via System::ExecutionInstanceGUID so every row loaded in one run shares a traceable identifier.
-

4.6 Incremental Load Strategy

Change Detection Approach

For each of the 14 tables, a **Lookup Transformation** compares incoming rows (from transformation.<Table>_Transformed) against the corresponding dw table using the table's Primary/Surrogate Key as the join condition (e.g., CustomerKey, ProductKey, SalesKey). This directly implements the project brief's suggested "Primary Key comparison" technique.

Insert Strategy

Rows with **no match** in dw (new keys) are routed via the Lookup's *No Match Output* directly to an OLE DB Destination performing a standard bulk INSERT into the target dw table.

Update Strategy

Rows with a **match** in dw (existing keys) are routed via the Lookup's *Match Output* to an OLE DB Command executing a parameterized UPDATE ... WHERE <Key> = ? statement per row, overwriting all business columns plus a freshly-regenerated LoadDate.

For the large Fact tables (FactSales: 3.4M rows, FactOnlineSales: 12.5M rows), row-by-row OLE DB Command updates were identified as a performance risk; a set-based MERGE statement approach via Execute SQL Task was adopted instead for these tables to avoid the row-by-row overhead (see Section 4.9 — Performance Optimization).

Duplicate Prevention Mechanism

Because the Lookup match is keyed on the table's Primary Key, a row can only ever be routed to exactly one of the two outputs (Insert or Update) — an existing key is always updated in place, never re-

inserted, which structurally prevents duplication regardless of how many times the pipeline is re-run against the same source data.

4.7 Challenges

1. **Source nature mismatch:** The provided source (ContosoRetailDW) is itself a pre-built dimensional warehouse rather than an OLTP system. This was addressed by explicitly treating its tables as the operational source and building an independent multi-layer pipeline around them, documented transparently rather than presented as raw operational data.
2. **Out-of-memory failure during parallel Fact validation:** Running all 4 Fact validation Data Flows fully in parallel caused a buffer allocation failure on FactOnlineSales (9M+ rows) due to combined memory pressure from FactSales (3.4M) and FactInventory (8M) running simultaneously. Resolved by adjusting the parallel execution pattern to reduce concurrent memory load.
3. **Data type mismatch on auto-generated staging columns:** BirthDate in DimCustomer was auto-generated as nvarchar rather than date when the staging table was created via SSIS's destination auto-generation, causing a business-rule expression failure. Resolved with an explicit (DT_DBTIMESTAMP) cast in the SSIS expression.
4. **Nullable-by-design columns found always populated:** Data profiling (aggregate NULL-count queries) revealed several columns declared nullable in the source schema (e.g., DiscountAmount, ReturnAmount in FactSales; several columns in FactOnlineSales/FactInventory) contained zero NULLs in practice. These were reclassified as mandatory, simplifying validation logic by removing unnecessary NULL-guards.
5. **Phantom/mismatched columns in the dw schema:** Several dw table columns (e.g., DimStore.WeeklyHours, StoreLabel, StoreManagementType) were defined during schema design but had no corresponding source column in the transformation layer, causing silent <ignore> mappings. Resolved by writing a systematic cross-schema comparison query and removing the unmatched columns.
6. **Column length overflow:** Audit columns (PackageName, ETLUser, BatchID) were initially sized too narrowly in the dw schema (e.g., NVARCHAR(8)), causing truncation risk on real values. Resolved by widening these columns uniformly across all 14 tables.
7. **Foreign Key-aware Truncate ordering:** Truncating dw tables required the reverse of the load order (Facts before Dimensions, and dependent Dimensions like DimProductSubcategory/DimStore/DimCustomer before their parent Dimensions), which took several iterations to get fully correct due to the multi-level dependency chain (e.g., DimProduct depends on DimProductSubcategory, which depends on DimProductCategory).
8. **Near-empty columns in the source data:** Systematic NULL-percentage profiling across the transformation schema revealed columns such as Title, Suffix (DimCustomer, ~99.5–99.98% NULL) and StopSaleDate, ImageURL, ProductURL (DimProduct, 100% NULL). These were excluded from the final dw schema on the basis that a column populated in under 5% of records

provides negligible analytical value, while columns like DimStore.CloseDate/CloseReason were deliberately retained despite similarly high NULL ratios because they directly support the StoreOperationalStatus business classification.

4.8 Conclusion

This project implemented a complete, five-layer Enterprise ETL pipeline (Staging → Validation → Business Transformation → Full Load → Incremental Load) using SQL Server Integration Services, processing over 20 million records across 4 Fact tables and 10 Dimension tables sourced from the ContosoRetailDW database. The solution applies a consistent three-stage data validation methodology, systematic data-driven design decisions (informed by explicit NULL-profiling rather than schema assumptions alone), a documented audit trail on every record, and a Lookup-based Incremental Load mechanism supporting safe re-execution without duplication.

Several real-world engineering challenges were encountered and resolved during development — including memory-constrained parallel execution, data type inconsistencies, and multi-level foreign key dependency ordering — each of which is documented above as evidence of the iterative, verification-driven approach taken throughout the build.

Recommendations for Future Improvements

- **Real-time / near-real-time ETL:** Replace the current batch (Full/Incremental) pattern with a Change Data Capture (CDC) or streaming-based approach for lower data latency.
- **Cloud integration:** Migrate the pipeline to a cloud-native platform (e.g., Azure Data Factory + Azure Synapse) for elastic compute, removing the memory constraints encountered during local parallel Fact processing.
- **Dashboards:** Build a reporting layer (Power BI or similar) directly on top of the dw schema to surface the implemented business classifications and financial calculations.
- **Automated monitoring:** Extend the existing Pipeline_Errors and validation summary logging into a scheduled SQL Server Agent Job with email alerting on failure or on Validation Success Rate falling below a defined threshold.